STEP 1 — Import Required Libraries

```
# Basic libraries for data handling
import numpy as np
import pandas as pd

# Load ready-made NLP datasets
from datasets import load_dataset

# Import BERT tokenizer and model
from transformers import BertTokenizerFast, BertForSequenceClassification

# Trainer utilities for training and evaluation
from transformers import Trainer, TrainingArguments

# Metrics for model evaluation
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusio

# Libraries for visualization
import matplotlib.pyplot as plt
import seaborn as sns

# PyTorch backend
import torch
```

STEP 2 — Load and Explore Dataset

```
# Load IMDb movie reviews dataset
dataset = load_dataset("imdb")

# Print dataset structure
print(dataset)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https:
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public mod
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
# Display one sample from training data
```

```
dataset["train"][0]
```

```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the
controversy that surrounded it when it was first released in 1967. I also heard that
at first it was seized by U.S. customs if it ever tried to enter this country,
therefore being a fan of films considered "controversial" I really had to see this for
myself.<br /><br />The plot is centered around a young Swedish drama student named
Lena who wants to learn everything she can about life. In particular she wants to
focus her attentions to making some sort of documentary on what the average Swede
thought about certain political issues such as the Vietnam War and race issues in the
United States. In between asking politicians and ordinary denizens of Stockholm about
their opinions on politics, she has sex with her drama teacher, classmates, and
married men.<br /><br />What kills me about I AM CURIOUS-YELLOW is that 40 years ago,
this was considered pornographic. Really, the sex and nudity scenes are few and far
between, even then it\'s not shot like some cheaply made porno. While my countrymen
mind find it shocking, in reality sex and nudity are a major staple in Swedish cinema.
Even Ingmar Bergman, arguably their answer to good old boy John Ford, had sex scenes
in his films.<br /><br />I do commend the filmmakers for the fact that any sex shown
in the film is shown for artistic purposes rather than just to shock people and make
money to be shown in pornographic theaters in America. I AM CURIOUS-YELLOW is a good
film for anyone wanting to study the meat and potatoes (no pun intended) of Swedish
cinema. But really, this film doesn\'t have much of a plot.',
 'label': 0}
```

```
# Check class distribution to see if dataset is balanced
labels = dataset["train"]["label"]
pd.Series(labels).value_counts()
```

	count
0	12500
1	12500

dtype: int64

STEP 3 — Load Tokenizer (Text Preprocessing)

```
# Load pretrained BERT tokenizer
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")
```

```
# Function to tokenize text
def tokenize_function(example):
    return tokenizer(
        example["text"],          # input text
        padding="max_length",     # pad all sequences to same length
        truncation=True,          # cut long texts
        max_length=256            # fixed sequence length
    )
```

```
# Apply tokenization to entire dataset
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

Map: 100% 25000/25000 [00:32<00:00, 929.49 examples/s]

STEP 4 — Prepare Dataset for PyTorch

```
# Rename label column to 'labels' (required by Hugging Face Trainer)
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")
```

```
# Convert dataset into PyTorch tensors
tokenized_dataset.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "labels"]
)
```

STEP 5 — Train–Test Split

```
# IMDb already has predefined splits
train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]
```

STEP 6 — Load Pretrained Transformer Model

```
# Load pretrained BERT model for classification
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",  # pretrained BERT
    num_labels=2          # binary classification
)
```

```
Loading weights: 100%      199/199 [00:00<00:00, 446.54it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok
- MISSING   :those params were newly initialized because missing from the checkpoint.
```

STEP 7 — Define Training Configuration

```
# Set training parameters
training_args = TrainingArguments(
    output_dir="./results",              # folder to save results
    eval_strategy="epoch",         # evaluate after each epoch
    learning_rate=2e-5,                  # standard learning rate for BERT
    per_device_train_batch_size=8,       # batch size for training
    per_device_eval_batch_size=8,        # batch size for evaluation
    num_train_epochs=2,                  # number of epochs
    weight_decay=0.01,                   # regularization
    logging_dir="./logs"                 # folder for logs
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGI
```

STEP 8 — Define Evaluation Metrics

```
# Function to calculate evaluation metrics
def compute_metrics(pred):
    labels = pred.label_ids
    predictions = np.argmax(pred.predictions, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )

    accuracy = accuracy_score(labels, predictions)

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

STEP 9 — Initialize Trainer and Train Model

```
# Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

```
# Train the model
trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarnin
  super().__init__(loader)
```

[85/6250 31:10 < 38:35:11, 0.04 it/s, Epoch 0.03/2]

Epoch	Training Loss	Validation Loss

STEP 10 — Model Evaluation

```
# Evaluate model on test data
results = trainer.evaluate()
print(results)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarnin
  super().__init__(loader)
```

[5/3125 00:57 < 12:28:26, 0.07 it/s]

STEP 11 — Confusion Matrix

```
# Get predictions on test dataset
predictions = trainer.predict(test_dataset)

# Convert predictions to class labels
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids
```